

Object Oriented Programming in C++

Constructors and destructors, operator
overloading, inheritance

Sushree Padhan

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

March 9, 2026

C++: Constructors and Destructors

- Constructors and destructors are special member functions: automatically called when objects are created and destroyed.

Constructor:

- A constructor is a member function whose task is to initialize the object's data members of its class.
- The constructor is automatically called when an object of its associated class is created.
- It has the same name as the class name and no return type.

```
1  class integer
2  {
3  int m, n;
4  public:
5  integer(void); // constructor declared
6  . . . . .
7  };
8  integer :: integer(void) // constructor defined
9  {
10 m = 0; n = 0;
11 }
```

C++: Constructors

```
1 integer int1; // object int1 created
```

- It creates the object `int1` of integer type `integer` and initializes its data members `m` and `n` to zero. There is no need to write any statement to invoke the constructor function.
- A constructor that accepts no parameters is called the **default constructor**.

```
1 integer() {}; // default constructor
```

- Example: for **class A**, let the constructor is **A::A()** is defined, then the compiler automatically provides a **default constructor** if no constructor is defined by the programmer. So, **A a;** invokes the default constructor of the compiler to create the object `a`.

C++: Constructors

Special characteristics of a Constructor function:

- They usually declared in the public section.
- They are invoked automatically when the objects are created.
- They do not have return types, not even void and therefore, and they cannot return values.
- Constructors are not inherited, but a derived class can call the base class constructor during object creation.
- Like other C++ functions, they can have default arguments.
- Constructors cannot be called like normal functions, and you cannot take their address.

Note: When a constructor with parameters is declared for a class and no default constructor is provided, initialization of the class objects becomes mandatory.

C++: Constructors

Parameterized constructor:

- The constructors that can take arguments are called parameterized constructors.

```
1  class integer {  
2  int m, n;  
3  public:  
4  integer(int x, int y); // parameterized constructor  
5  . . . . .  
6  . . . . .  
7  };  
8  integer :: integer(int x, int y)  
9  {  
10 m = x; n = y;  
11 }
```

- int main() { integer int1; }** object declaration statement do not work here.
- Pass the initial values as arguments to the constructor function when an object is declared. This can be done in two ways:
 - ▶ By calling the constructor explicitly.
 - ▶ By calling the constructor implicitly.

C++: Constructors

Parameterized constructor:

```
1 integer int1 = integer(0,100); // explicit call
2 // creates an integer object int1 and passes the values 0
  and 100 to it. It is less common but still valid.
```

```
1 integer int1(0,100); // implicit call
2 // It is the preferred and commonly used form.
```

- **Inline constructor:** Constructor functions can also be defined as inline functions.
- The parameters of a constructor can be of any type. However, if the parameter is of the same class type, it must be passed by reference.

Copy constructor: can accept a reference to its own class as a parameter.

```
1 class A {
2     . . . . .
3     public:
4     A(A); }; //illegal
5     Class B {
6         . . . . .
7         public:
8         B(B &); }; // (called as copy constructor) legal
```

C++: Constructors

Copy constructor:

- A copy constructor takes a reference to an object of the same class and creates a copy of it.

```
1  class Student {
2  int id; int rollno;
3  public:
4  // Normal constructor
5  Student(int i, int j) {
6  id = i; rollno=j; }
7  // Copy constructor
8  Student(Student &s) {
9  id = s.id; // rollno = s.rollno;
10 }
11 void display() {
12 cout << "ID: " << id << ", roll no: " << rollno << endl; } };
13 int main() {
14 Student s1(101,2023005);    // Normal constructor
15 Student s2 = s1;           // Copy constructor called
16 s1.display();
17 s2.display();
18 return 0;}
```

Output:

ID: 101, roll no: 2023005

ID: 101, roll no: 0

C++: Constructors

- **Default Copy constructor:**

```
1  #include<iostream>
2  using namespace std;
3  class Student {
4  int id; int rollno;
5  public:
6  // Normal constructor
7  Student(int i, int j) {
8  id = i; rollno=j; }
9  void display() {
10 cout << "ID: " << id << ", roll no: " << rollno << endl; } };
11 int main() {
12 Student s1(101,2023005);    // Normal constructor
13 Student s2 = s1;           // Default Copy constructor called
14 s1.display();
15 s2.display();
16 return 0;}
```

Output:

ID: 101, roll no: 2023005

ID: 101, roll no: 2023005

C++: Constructors

Copy Assignment Operator: assigns the contents of one existing object to another existing object.

```
1  class Student {
2  int id; int rollno;
3  public:
4  Student() { id = 0; rollno = 0; } // Default constructor
5  // Normal constructor
6  Student(int i, int j) {
7  id = i; rollno=j; }
8  // Copy assignment operator
9  Student& operator=(Student &other) {
10 id = other.id;
11 rollno = other.rollno;
12 return *this; }
13 void display() {
14 cout << "ID: " << id <<"", roll no: "<<rollno<< endl; } };
15 int main() {
16 Student s1(101,2023005); // Normal constructor
17 Student s2;
18 s2=s1; // copy assignment operator is called.
19 s2.display();
20 return 0;}
```

Output:

ID: 101, roll no: 2023005

C++: Constructors

Multiple constructors in a class:

```
1  class integer
2  {
3  int m, n;
4  public:
5  integer() { m=0; n=0; } // constructor 1
6  integer(int a, int b)
7  { m = a; n = b; } // constructor 2
8  integer(integer &i)
9  { m = i.m; n = i.n; } // constructor 3
10 };
```

```
1  integer I1; //invoke the first constructor
2  integer I2(20,40); //invoke the second constructor
3  integer I3(I2); //invoke the third constructor
4  // sets the value of every data element of I3 to the value
   of the corresponding data element of I2
```

- A copy constructor is used to create a new object as a copy of an existing object.
- When a class has multiple constructors with different parameters, it is called constructor overloading.

C++: Constructors

Constructors with default arguments:

```
1  class complex
2  {
3  float x, y;
4  public:
5  complex(float real, float imag=0);
6  };
7  complex::complex(float real, float imag)
8  {
9  x = real; y = imag;
10 }
```

- The default value of the argument imag is zero.

```
1  complex C(5.0);
```

- It assigns the value 5.0 to the real variable and 0.0 to imag.

```
1  complex C(2.0,3.0);
```

- It assigns 2.0 to real and 3.0 to imag. The actual parameter, when specified, overrides the default value.

C++: Constructors

Dynamic initialization:

```
1  class Number
2  {
3  int x;
4  public:
5  // Constructor
6  Number(int a) {
7  x = a; }
8  void show() {
9  cout << "Value of x = " << x; }
10 };
11 int main()
12 {
13 int n;
14 cout << "Enter a number: ";
15 cin >> n; // Dynamic initialization
16 Number obj(n); // value of x decided at runtime
17 obj.show();
18 return 0;
19 }
```

C++: Constructors

Dynamic constructors:

- Allocation of memory to objects at the time of their construction is known as dynamic construction of objects.
- This enables the system to allocate the right amount of memory for each object when the objects are not of the same size, thus saving memory.

```
1  class A {
2  int *arr;
3  public:
4  // Dynamic constructor
5  A(int size) {
6  arr = new int[size];    // dynamic memory allocation
7  for (int i = 0; i < size; i++) {
8  arr[i] = i + 1;}  }
9  void display(int size) {
10 for (int i = 0; i < size; i++) {
11 cout << arr[i] << " ";}  }
12 };
13 int main() {
14 A a(5);    // constructor called dynamically
15 a.display(5);
16 return 0; }
```

C++: Destructors

- A **destructor** is automatically called to destroy an object.
- It is a member function whose name is the same as the class name but is preceded by a tilde.
- Example:

```
1 ~A(){ }  
2 ~integer(){ }
```

- A destructor never takes any argument nor does it return any value.
- It will be invoked implicitly by the compiler when an object is destroyed or goes out of scope to clean up storage that is no longer accessible.
- Whenever new is used to allocate memory in the constructors, it should be released using delete to free that memory.

C++: Destructors

```
1  class A {
2  int *arr;
3  public:
4  // Dynamic constructor
5  A(int size) {
6  arr = new int[size];    // dynamic memory allocation
7  for (int i = 0; i < size; i++)
8  {
9  arr[i] = i + 1;
10 } }
11 void display(int size) {
12 for (int i = 0; i < size; i++)
13 {
14 cout << arr[i] << " ";
15 } }
16 // Destructor to free memory
17 ~A() {
18 delete[] arr;
19 } };
20 int main() {
21 A a(5);    // constructor called dynamically
22 a.display(5);
23 return 0; }
```

C++: Operator Overloading

- Operator overloading allows redefining the behavior of operators for user-defined types.
- It is feature to give **custom meaning to built-in operators** when they are **used with user-defined types**.
- We can overload (give additional meaning to) all the C++ operators except:
 - Class member access operators (., .*)
 - Scope resolution operator (::)
 - Size operator (sizeof)
 - Conditional operator (?:)
- Why we cannot overload these operators? These operators cannot be overloaded because they are essential to the core syntax and semantics of the C++ language, and their behavior must remain fixed for the compiler to interpret programs correctly.

C++: Operator Overloading

- When an operator is overloaded, its original meaning is not lost. Example: the operator `+`, which has been overloaded to add two vectors, can still be used to add two integers.

Defining **operator overloading**:

- To define an additional behavior for an operator, we must specify how it operates on objects of a class.
- This is done by defining a special member function using the **operator** keyword.

```
1 return_type class_name :: operator op(arglist)
2 {
3     // statements
4 }
```

- Return type is the type of value returned by the specified operation and **op** is the operator being overloaded.
- Operator overloading functions** can be **member functions** or **non-member (global) functions**. A **non-member functions** are often declared as friend functions to access private members.

C++: Operator Overloading

- Operator functions are declared in the class using prototypes such as:

```
1 MyClass operator+(const MyClass&, const MyClass&); //global
2 MyClass operator+(const MyClass&); //member
3 friend MyClass operator+(const MyClass&, const MyClass&);
  //friend
4
5 MyClass operator++(const MyClass&); //global
6 MyClass operator++(); //member
7 friend MyClass operator++(const MyClass&); //friend
```

Overloading examples: MyClass a,b;

- operator+(a,b)** function means **a+b** expression and **operator++(a)** means **++a** in **global or friend** functions.
- a.operator+(b)** means **a+b** and **a.operator++()** means **++a** in **member** functions.

C++: Operator Overloading

Overloading Unary Operators:

- Example (unary minus): The unary minus when applied to an object should change the sign of each of its data items.

```
1  class space {
2  int x; int y;
3  public:
4  void getdata(int a, int b);
5  void display(void);
6  void operator-(); // overload unary minus
7  };
8  void space :: getdata(int a, int b) {
9  x = a; y = b; }
10 void space :: display(void) {
11 cout<< "x = "<<x<<" ";
12 cout<<"y = "<<y<<"\n";}
13 void space :: operator-() {
14 x = -x; y = -y; }
15 int main() {
16 space S;
17 S.getdata(10, -20); cout << "S : "; S.display();
18 -S; // activates operator-() function
19 cout << "-S : "; S.display();
20 return 0; }
```

C++: Operator Overloading

Overloading Unary Operators:

- Let's say the class has two objects: sample S1, S2; S2 = -S1; this will not work because the function operator-() does not return any value. It can work if the function is modified to return an object.
- It is possible to overload a unary minus operator using a friend function.

```
1 friend space operator-(const space &s); // declaration
2 space operator-(const space& s)
3 {
4     space temp;
5     temp.x = -s.x;
6     temp.y = -s.y;
7     return temp;
8 }
```

```
1 space operator-(); // declaration
2 space space::operator-() // definition
3 {
4     x = -x; y = -y;
5     return *this;
6 }
```

C++: Operator Overloading

Overloading Binary Operators:

- Example using member function (for binary plus):

```
1  class complex {
2  float x; //real part
3  float y; // imaginary part
4  public:
5  complex(){ } //constructor 1
6  complex(float real, float imag) //constructor 2
7  { x = real; y = imag; }
8  complex operator+(const complex& c);
9  void display(); };
10 complex complex :: operator+(const complex& c) {
11 complex temp; //temporary
12 temp.x = x + c.x; // these are
13 temp.y = y + c.y; //float additions
14 return(temp); }
15 void complex :: display() {
16 cout << x << " + j" << y << "\n"; }
17 int main() {
18 complex C1, C2, C3; //calls constructor 1
19 C1 = complex(2.5,3.5);C2 = complex(1.6,2.7); //constructor 2
20 C3 = C1 + C2; //like as function call C3 = C1.operator+(C2);
21 cout << "C1 = "; C1.display();
22 cout << "C2 = "; C2.display();
23 cout << "C3 = "; C3.display(); return 0; }
```

C++: Operator Overloading

Overloading binary plus operator using friend function:

```
1  class complex {
2  float x; float y;
3  public:
4  complex(){ }
5  complex(float real, float imag)
6  { x = real; y = imag; }
7  friend complex operator+(const complex& a, const complex& b);
8  void display(); };
9  complex operator+(const complex& a, const complex& b)
10 //const ensures function does not modify objects. The compiler
    prevents accidental modification to the original numbers.
11 {
12 return complex((a.x+b.x),(a.y+b.y));
13 }
14 void complex :: display() {
15 cout << x << " + j" << y << "\n"; }
16 int main() {
17 complex C1, C2, C3; //invokes constructor 1
18 C1 = complex(2.5,3.5);C2 = complex(1.6,2.7); //constructor 2
19 C3 = C1 + C2; // equivalent as C3 = operator+(C1, C2);;
20 cout << "C1 = "; C1.display();
21 cout << "C2 = "; C2.display();
22 cout << "C3 = "; C3.display();
23 return 0; }
```

C++: Operator Overloading

Overloading binary plus operators using global function:

```
1  class complex {
2  private:
3  float x, y;
4  public:
5  complex() { x = 0; y = 0; }
6  complex(float real, float imag) {
7  x = real; y = imag;}
8  float getReal() const { //constant member function,
9  return x; }
10 float getImag() const { //does not modify the object.
11 return y;}
12 void display() {
13 cout << x << " + j" << y << endl; } };
14 complex operator+(const complex& a, const complex& b) {
15 float real = a.getReal()+b.getReal();
16 float imag = a.getImag()+b.getImag();
17 return complex(real, imag);}
18 int main() {
19 complex C1, C2, C3;
20 C1 = complex(2.5, 3.5); C2 = complex(1.6, 2.7);
21 C3 = C1 + C2;
22 cout << "C1 = "; C1.display();
23 cout << "C2 = "; C2.display();
24 cout << "C3 = "; C3.display(); return 0; }
```

C++: Operator Overloading

Overloading « operator:

```
1  class A {
2  public:
3  int x, y;
4  A(int a = 0, int b = 0) {
5  x = a;
6  y = b; }
7  friend ostream& operator<<(ostream& out, const A& p); };
8  ostream& operator<<(ostream& out, const A& p) {
9  out << "(" << p.x << ", " << p.y << ")";
10 //operator<<(out, "("); operator<<(out, p.x); operator<<(out, ",
    "); operator<<(out, p.y); operator<<(out, ")");
11 return out; } //Function returns a ref to cout.
12 int main() {
13 A p1(6,7);
14 A p2(3,9);
15 cout << p1 << endl; //operator<<(operator<<(cout,p1),endl);
16 //out=cout and p=p1
17 cout << p2 << endl;
18 return 0; }
```

Output:

(6, 7)

(3, 9)

C++: Operator Overloading

Overloading == operator:

```
1  class B {
2  public:
3  int x, y;
4  B(int a = 0, int b = 0) {
5  x = a;
6  y = b; }
7  bool operator==(const B& p) {
8  return (x == p.x && y == p.y);
9  } };
10 int main() {
11 B p1(2,3);
12 B p2(2,3);
13 B p3(4,5);
14 if (p1 == p2) // calls p1.operator==(p2) so this->p1 and p
    becomes p2. p1.x == p2.x && p1.y == p2.y
15 cout << "p1 and p2 are equal\n";
16 else
17 cout << "p1 and p2 are not equal\n";
18 if (p1 == p3)
19 cout << "p1 and p3 are equal\n";
20 else
21 cout << "p1 and p3 are not equal\n";
22 return 0; }
```

C++: Rules for Overloading Operators

- Only existing operators can be overloaded. New operators cannot be created.
- The overloaded operator must have at least one operand that is of user-defined type.
- Overloaded operators must follow the syntax and precedence rules of the original operators.
- There are some operators (scope resolution, member access, ternary conditional, sizeof) that cannot be overloaded.
- Unary operators, overloaded by means of a member function, take no explicit arguments, but, those overloaded by means of a friend function, take one reference argument (the object of the relevant class).
- Binary operators overloaded through a member function take one explicit argument and those which are overloaded through a friend function take two explicit arguments.
- When using binary operators overloaded through a member function, the left hand operand must be an object of the relevant class.
- Binary arithmetic operators such as +, -, *, and / usually return a new object and typically do not modify their operands.

C++: Inheritance

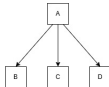
- **Resuability:** It allows us to reuse existing code instead of writing it again.
- Creating new classes, reusing the properties of the existing ones.
Inheritance: the mechanism of deriving a new class from an existing class.
- The **existing class** is referred to as the **base class** and the **new class** is called the **derived class** or subclass.
- The derived class inherits some or all of the traits from the base class. A class may also inherit from multiple classes or through multiple levels.



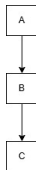
Single inheritance



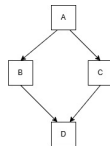
Multiple inheritance



Hierarchical inheritance



Multilevel inheritance



Hybrid inheritance

C++: Inheritance

Defining derived classes:

```
1 class derived-class-name : visibility-mode base-class-name
2 {
3     ....//
4     ....// members of derived class
5     ....//
6 };
```

```
1 class derived: private base //private derivation
2 {
3     members of derived
4 };
5 class derived: public base // public derivation
6 {
7     members of derived
8 };
9 class derived: base // private derivation by default
10 {
11     members of derived
12 };
```

C++: Inheritance

Defining derived classes:

- **Access specifiers:**
 - ▶ Derived class cannot access private members of a base class.
 - ▶ Derived class can access public members of a base class.
- **Constructor and destructor:**
 - ▶ A constructor of the derived class must first call the constructor of the base class to construct base class instance of the derived class.
 - ▶ When a derived class object is destroyed, the derived class destructor executes first, followed automatically by the base class destructor.
 - ▶ Execution order:
First base class constructor, then derived class constructor.
First, the derived class destructor is called, then the base class destructor is called automatically afterwards.
- Static members of a base class are inherited, but friend functions are not inherited.

C++: Inheritance

Protected access specifier:

- **Protected** specifier allows members of a class to be accessible within the class and its derived classes.
- Derived class can directly access protected member of the base class.
- A protected member of a base class behaves like a private member for outside classes, but it remains accessible to derived classes.

Example:

```
1  class Base {  
2  public:  
3  int x = 10;  
4  void showx() {  
5  cout << "x = " << x << endl; }  
6  protected:  
7  int y = 20;  
8  void showy() {  
9  cout << "y = " << y << endl; }  
10 private:  
11 int z = 30;  
12 void showz() {  
13 cout << "z = " << z << endl; } };
```

C++: Inheritance

Single Inheritance:

```
14 class PublicDerived : public Base {  
15     public:  
16     void display() {  
17         showx();    // public -> public  
18         showy();    // protected -> protected  
19         // showz(); // private not accessible  
20     } };
```

```
21 class ProtectedDerived : protected Base {  
22     public:  
23     void display() {  
24         showx();    // public -> protected  
25         showy();    // protected -> protected  
26     } };
```

```
27 class PrivateDerived : private Base {  
28     public:  
29     void display() {  
30         showx();    // public -> private  
31         showy();    // protected -> private  
32     } };
```

C++: Inheritance

Single Inheritance:

```
33 int main() {  
34     PublicDerived p1;  
35     p1.showx();           // allowed  
36     p1.display();  
37     ProtectedDerived p2;  
38     // p2.showx();       // not allowed  
39     p2.display();  
40     PrivateDerived p3;  
41     // p3.showx();       // not allowed  
42     p3.display();  
43     return 0; }
```

Output:

x = 10

x = 10

y = 20

x = 10

y = 20

x = 10

y = 20

C++: Inheritance

Multiple Inheritance:

```
1 class C: visibility A, visibility B
2 {
3     ....
4     ....(Body of C)
5     ....}
```

Multilevel Inheritance:

```
1 class A
2 { }; // Base class
3 class B: public A
4 { }; // B derived from A
5 class C: public B
6 { }; // C derived from B
```

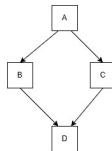
Hierarchical Inheritance:

```
1 class A { }; // Base class
2 class B: public A
3 { }; // B derived from A
4 class C: public A
5 { }; // C derived from A
6 class D: public A
7 { }; // D derived from A
```

C++: Inheritance

Hybrid Inheritance:

- A situation of multiple and multilevel inheritance.
- **Virtual base class:** shared my multiple derived classes so that one copy of it exists.



```
1 class A //grandparent
2 {
3 };
4 class B:public A //parent1
5 {
6 };
7 class C:public A //parent2
8 {
9 };
10 class D:public B, public C //
    child
11 {
12 .... //two copies of A will be
    inherited (ambiguous error)
13 };
```

```
1 class A // grandparent
2 {
3 };
4 class B:virtual public A //
    parent1
5 {
6 };
7 class C:public virtual A //
    parent2
8 {
9 };
10 class D:public B, public C //
    child
11 {
12 .... //only one copy of A will
    be inherited
13 };
```

C++: Inheritance

Constructors and Destructors in Derived class:

```
1 class A {
2 public:
3 A(int x) {
4 cout << "ConstructorA, x = " << x << endl;
5 }
6 ~A() {
7 cout << "DestructorA" << endl;} };
8 class B {
9 public:
10 B(int x, int y) {
11 cout << "ConstructorB, x = " << x << ", y = "
12 << y << endl; }
13 ~B() {
14 cout << "DestructorB" << endl;} };
15 class C : public A, public B {
16 public:
17 C(int x, int y, int z) : A(x), B(x, y) {
18 cout << "ConstructorC, z = " << z << endl;}
19 ~C() {
20 cout << "DestructorC" << endl;} };
21 int main() {
22 C obj(10, 20, 30);
23 return 0; }
```

Output:

ConstructorA, x = 10

ConstructorB, x = 10, y = 20

ConstructorC, z = 30

DestructorC

DestructorB

DestructorA

Any Questions?